Medical Journey Log: A Family Care Tracker

Alex Frear

CST-452 Milestone 6: Final Project Completion

Grand Canyon University

Instructor: Professor Amr Elchouemi

11/25/2025

GitHub Repository: https://github.com/amfrear/CST452/tree/main/Milestone%206

Application Demo Video 1: Medical journey Log - Code Review

Application Demo Video 2 (Code Review): Medical Journey Log - Application Running

Portfolio Website: https://amfrear.github.io/MedicalJourneyLog/

Table of Contents

1. Executive Summary

The Medical Journey Log: A Family Care Tracker is a web application built using ASP.NET Core Razor Pages and MySQL to help caregivers document, monitor, and track children's medical symptoms, notes, milestones, and treatment history. The system was developed across CST-451 and CST-452 using an iterative Software Development Life Cycle (SDLC) approach. The project leveraged Planning, Requirements Analysis, Design Modeling, Implementation, Testing, and Deployment phases to deliver a complete, functional, and stable system meeting 100% of the defined requirements.

Throughout development, the system evolved from a basic child-profile logger into a fully functional caregiver tool with CRUD operations, validation, guarded workflows, and complete traceability across requirements, design, code, and tests. All iterations were validated through unit, integration, system, and regression testing, and the final iteration confirmed the stability and correctness of every feature.

This Final Project Review consolidates the results of the prior milestones, summarizes system functionality, evaluates requirement completion, reflects on lessons learned, and presents recommendations for future enhancements.

2. Final Requirements Review

The following table summarizes **all functional requirements** originating from Milestone 2 (Requirements Analysis Phase) and includes their final completion status. Each requirement is marked as **Met**, **Partially Met**, or **Not Met**, with supporting evidence from implementation and testing.

2.1 Requirements Completion Summary Table

Functional Requirements

Req ID	Requirement Description	Status	Evidence	Future Improvements
FR1	The system shall allow a caregiver to add and manage child profiles.	**Met**	AddChild, EditChild, DeleteChild (guarded). Displayed on Index and ChildDetails pages.	Additional child fields, profile photos.
FR2	The system shall display a list of all children on the home page.	**Met**	Index page displays all children with navigation links.	Sorting, filtering, or search.
FR3	The system shall allow caregivers to view detailed information for a selected child.	**Met**	ChildDetails page shows full child record and symptoms.	Pagination or graph summaries.
FR4	The system shall allow caregivers to log symptoms for a child.	**Met**	LogSymptom page; validated form fields.	Add severity levels or categories.
FR5	The system shall allow caregivers to edit existing symptom entries.	**Met**	EditSymptom implemented; validated and tested.	Add edit history or notes.
FR6	The system shall allow caregivers to delete symptom entries.	**Met**	DeleteSymptom workflow fully functional; confirmed in testing.	Optional soft-delete recovery.
FR7	The system shall prevent deletion of a child if symptoms exist.	**Met**	Guard logic prevents child deletion until symptoms are removed.	Provide “delete all symptoms” option.
FR8	The system shall provide validation for required inputs.	**Met**	Data annotations enforce required fields, lengths, and formats.	Expand validation rules; client-side JS.
FR9	The system shall store all application data in a relational database.	**Met**	MySQL schema with Child/Symptom tables and foreign keys.	Add indexing; optimize queries.

Non-Functional Requirements

Req ID	Description	Status	Evidence	Future Improvements
NFR1	The system shall use a relational database with structured, normalized tables.	**Met**	MySQL Child and Symptom tables, foreign key constraint.	Add indexing for large datasets.
NFR2	The system shall provide a clean and consistent user interface.	**Met**	Razor Pages layout, Showcase poster UI screenshots.	Improve mobile responsiveness.
NFR3	The system shall follow SDLC documentation including requirements, design, implementation, and testing.	**Met**	Milestones 1–6 completed; poster included.	Add more formal UML diagrams.
NFR4	The system shall include responsive input validation and user feedback.	**Met**	ModelState validation; success/failure alerts on pages.	Add toast notifications.
NFR5	The system shall protect against accidental data loss through guarded workflows.	**Met**	Child delete blocked when symptoms exist; confirmation pages used.	Add optional data backup/export.

2.2 Requirements Completion Summary

- **Total Requirements:** 15
- **Met:** 15
- **Partially Met:** 0
- **Not Met:** 0

All requirements defined in the initial Analysis Phase were fully implemented, validated, and demonstrated across Milestones 4 and 5. The project meets 100% of functional and non-functional requirements.

3. Design Review

The design of the Medical Journey Log application was established during the CST-451 Milestone 3 Architectural Plan and served as the blueprint for all development work in CST-451 and CST-452. The system was designed using a layered architecture with Razor Pages for the presentation layer, C# page models for business logic, and MySQL as the relational data store. This structure provided clear separation of concerns and ensured maintainability, scalability, and traceability throughout the project.

3.1 System Architecture Overview

The system architecture follows a standard multi-tier structure:

- **Presentation Layer:** Razor Pages (.cshtml) used to render views and collect caregiver input.
- **Logic Layer:** Page Model files (.cshtml.cs) responsible for validation, data loading, and handling form submissions.
- **Data Layer:** A MySQL database storing all persistent data for children and symptom entries.
- **Identity Layer (Not Implemented):** Authentication was intentionally excluded because the system is designed for single-device, personal caregiver use.

This architecture, originally defined in Milestone 3, supported consistent development and simplified testing and debugging across all milestones.

3.2 Database Schema

The database schema is relational and centered on two core entities: **Child** and **Symptom**. The design enforces a one-to-many relationship, where each child can have multiple symptoms, but each symptom is linked to only one child.

Child Table Fields:

- ChildId (PK)
- Name
- DateOfBirth

Symptom Table Fields:

- SymptomId (PK)
- ChildId (FK)
- Name

- Description
- DateLogged

This schema was validated through development and testing and supports guarded behaviors such as preventing deletion of a child with existing symptom records.

3.3 Navigation Flow

The application's navigation flow was defined in Milestone 3 and carried through into the final build:

1. **Home Page (Index):** Displays all children and provides navigation to add, view, or manage records.
2. **Child Details Page:** Shows detailed child information and all associated symptoms with options to edit or delete entries.
3. **Add/Edit Child Pages:** Used to create or update child profiles with server-side validation.
4. **Log Symptom / Edit Symptom Pages:** Allow caregivers to add or modify symptom entries.
5. **Delete Confirmation Pages:** Provide guarded confirmation dialogs before removing data.

The implemented system matches this planned navigation structure and supports all required workflows.

3.4 User Interface Wireframes

Wireframes created in Milestone 3 guided the layout of all final Razor Pages, including:

- Child profile screens
- Symptom logging forms
- Detail views
- Edit and delete screens

The final UI closely aligns with these wireframes, maintaining a simple, accessible, and caregiver-friendly layout. The final screens shown in the Technology Showcase Poster reflect this alignment.

3.5 Design Evaluation

The system design proved effective in supporting the full implementation:

- Architecture provided clear separation of responsibilities.
- Database design ensured reliable relational data storage.
- Wireframes and navigation plans translated directly into completed pages.
- All functional and non-functional requirements mapped cleanly back to the design artifacts.

The design phase established a strong foundation that allowed development and testing to progress without major refactoring or architectural changes.

4. Implementation Review

The implementation of the Medical Journey Log application occurred across three major iterations completed during CST-451 and CST-452. Iteration 1 established core functionality, Iteration 2 expanded CRUD capabilities and validation, and Iteration 3 focused on regression testing, UI polish, and documentation. The final system meets 100% of functional and non-functional requirements and reflects the full SDLC planned during earlier milestones.

4.1 Iteration 1: Core Functionality

Iteration 1, completed during CST-451 Milestone 4, focused on establishing the foundational caregiver workflow. The following features were implemented:

- Creation of child profiles with name and date of birth
- Logging symptoms for a selected child
- Displaying all symptoms associated with a child
- Navigation from the homepage to child details and symptom logging
- MySQL database configuration and initial schema setup
- Successful manual testing of all implemented features

This iteration delivered the initial end-to-end workflow that allowed caregivers to enter and review medical information for each child.

4.2 Iteration 2: CRUD Expansion and Validation

Iteration 2, completed in CST-452 Milestone 4, expanded the system to include full create, read, update, and delete operations for both children and symptoms. The focus was on deeper functionality, data integrity, and validation. The following enhancements were implemented:

- **Edit Child**: Ability to update child names and dates with validation rules
- **Delete Child (Guarded)**: Children with existing symptoms cannot be deleted
- **Edit Symptom**: Ability to correct previously logged symptoms
- **Delete Symptom**: Removal of individual symptom entries
- **Validation Enforcement**: Data annotations to prevent empty fields, overlong input, or invalid entries
- **Improved User Feedback**: Success and error alerts included on UI screens

This iteration ensured that caregivers could manage data accurately while maintaining database consistency and preventing accidental data loss.

4.3 Iteration 3: Regression Testing and UI Polish

Iteration 3, completed during CST-452 Milestone 5, focused on quality assurance and preparing the system for final submission. No new features were added; instead, the emphasis was on stability and correctness. This iteration included:

- Regression testing of all CRUD workflows
- UI review for consistency, readability, and alignment across pages
- Verification of validation behavior and guarded delete logic
- Code cleanup, repository organization, and documentation updates
- Completion of demonstration screencasts showcasing code and application functionality

All features from Iterations 1 and 2 passed regression testing with no defects, confirming that the application is stable and production-ready within the scope of the project.

4.4 Source Code Summary

The final implementation includes the following key components:

- Razor Pages for all functional screens (Create, Edit, Delete, Details, Index)
- C# Page Models handling GET/POST actions, validation, and data persistence
- MySQL relational schema storing Child and Symptom records
- Data validation through C# model annotations
- Guarded workflow logic preventing unsafe deletions
- Centralized navigation layout for a consistent caregiver user experience

The codebase is organized, commented, and fully traceable to requirements defined in Milestone 2, design artifacts from Milestone 3, and testing conducted in Milestones 4 and 5.

5. Testing Review

Testing for the Medical Journey Log application was conducted across three major iterations and included unit testing, integration testing, system testing, and full regression testing. These tests validated all functionality, ensured data integrity, confirmed guarded behaviors, and demonstrated compliance with the system's functional and non-functional requirements. By the end of Iteration 3, all test cases passed successfully, and the application met 100% of all defined requirements.

5.1 Testing Strategy Overview

The testing approach followed a structured, multi-level process:

- **Unit Testing:**
  Verified individual components such as model validation rules, form submissions, and page model logic.
- **Integration Testing:**
  Focused on ensuring that application components—UI pages, page model handlers, and the MySQL database—worked together correctly.
- **System Testing:**
  Validated complete caregiver workflows, including adding children, logging symptoms, editing entries, and deleting records.
- **Regression Testing:**
  Conducted in Iteration 3 to confirm that all functionality implemented in Iterations 1 and 2 continued to perform without defects after updates, UI polish, and code refinements.

This layered approach ensured stable and predictable application behavior across the entire development lifecycle.

5.2 Unit Testing Summary

Unit testing focused on validating:

- Required fields (e.g., child name, symptom name)
- Maximum length constraints
- Correct handling of invalid input
- Redirect behavior after successful form submissions
- Guard logic and business rules (e.g., preventing child deletion with symptoms)

All unit tests passed, confirming that individual components behaved as expected and enforced validation rules consistently.

5.3 Integration Testing Summary

Integration tests verified how well the Razor Pages, Page Models, and MySQL database interacted. Tests included:

- Creating a child and verifying the record in the database
- Logging symptoms and confirming they appeared on the Child Details page
- Editing and deleting symptoms while maintaining referential integrity
- Attempting to delete a child with linked symptoms

Integration testing confirmed that:

- All forms correctly persisted data to MySQL
- Page routing and handlers executed correctly
- Guarded delete logic behaved as designed
- All CRUD operations functioned consistently across the UI and database layers

All integration test cases passed.

5.4 System Testing Summary

System testing validated full caregiver workflows end-to-end. Test cases included:

- Adding a child and logging multiple symptoms
- Editing and deleting child records
- Editing and deleting symptom records
- Navigating between pages using UI links
- Confirming validation messages appeared when required

These tests confirmed that the user experience matched the intended design and system requirements. All system tests passed successfully.

5.5 Regression Testing Summary

Regression testing was performed during Iteration 3 (Milestone 5) to ensure that recent updates, UI improvements, and code cleanup did not introduce new defects. This included re-running all test cases from Iterations 1 and 2:

- All **unit tests** passed with no regressions
- All **integration tests** passed and confirmed stable database interactions

- All **system tests** confirmed that end-to-end workflows operated correctly
- Guard logic for child deletion remained fully functional
- Validation messages worked consistently across edit and create pages

Regression testing confirmed complete stability and consistency across all modules.

5.6 Final Testing Outcome

The final testing results show:

- **Total Unit Test Cases:** 8 – **Passed: 8 / Failed: 0**
- **Total Integration Test Cases:** 4 – **Passed: 4 / Failed: 0**
- **Total System Test Cases:** 3 – **Passed: 3 / Failed: 0**
- **Guarded Workflows:** 100% functioning as expected
- **Validation Rules:** 100% enforced correctly

The application meets all test objectives, validates all requirements, and demonstrates full readiness for final submission.

6. Security & Risk Review

Security and risk management were considered throughout the development of the Medical Journey Log application, with a primary focus on protecting sensitive medical information, preventing accidental data loss, and maintaining data integrity. Because the system is intended for *personal, local use by one caregiver or two shared family members*, the application does not implement cloud storage, multi-user authentication, or remote access. Instead, security decisions were centered around simplicity, controlled local access, and safe data handling within the scope of the project.

6.1 Access and Usage Context

The application is designed as a **local, personal-use tool**, intended to be run directly on a caregiver's computer within a trusted household environment. Earlier project milestones defined this clearly:

- No multi-user authentication
- No cloud connectivity
- No role-based access control
- Entire system runs locally on macOS using a local MySQL database

Security decisions were therefore scoped to protect data integrity and minimize risks within this personal-use context.

6.2 Data Validation and Input Handling

To maintain application safety and prevent invalid or harmful input, the system implements strict server-side validation using C# data annotations. These include:

- Required fields (e.g., child name, symptom name)
- Maximum length constraints to prevent oversized or malformed inputs
- Date validation where appropriate
- ModelState enforcement before saving to the database

Validation prevents incomplete or incorrect entries from being persisted and reduces the risk of bad data corrupting the system.

6.3 Data Integrity and Guarded Workflows

Several guarded behaviors were implemented to protect caregivers from accidental data loss:

- **A child cannot be deleted if associated symptoms exist.**
  This prevents losing medical history by mistake.
- **Symptoms must be intentionally deleted one at a time** before a child can be removed.
- **Delete pages use confirmation prompts** to ensure users do not remove records unintentionally.

These protections reinforce data integrity and ensure the relational structure of the MySQL database remains consistent.

6.4 Local Database and Storage Considerations

The application uses a **local MySQL database**, which provides controlled access and avoids exposure to outside networks. Security benefits of local-only design include:

- No external attack surface
- No remote connections
- All data stored privately on the user's machine
- Relational constraints (primary and foreign keys) maintain structural integrity

The SQL schema confirms properly structured relationships between tables and field-level constraints.

6.5 Application Security Risks and Mitigation

Although the application is local and does not include authentication, several risks were identified and mitigated:

Risk: Unauthorized Access on the Local Machine

- Mitigation: Users are expected to run the application on a personal device secured through operating system accounts (e.g., macOS user login).

Risk: Accidental Deletion or Data Loss

- Mitigation: Guarded delete logic, confirmation pages, and structured workflows.

Risk: Invalid or problematic input

- Mitigation: Strong server-side validation prevents malformed or incomplete data.

Risk: Data Corruption

- Mitigation: MySQL relational constraints enforce referential integrity.

Risk: Local Environment Misconfiguration

- Mitigation: Detailed setup instructions ensure proper database and application configuration.

6.6 Overall Security Assessment

Within the project's defined scope, the system meets all necessary security expectations:

- Entire application runs in a trusted local environment
- No external or cloud exposure
- Strong validation ensures safe input handling
- Guarded workflows prevent unintended data loss
- Database schema enforces structural consistency

While future versions could incorporate user authentication, encryption, and cloud synchronization, the current design aligns precisely with the project's goals and requirements as defined in earlier milestones.

7. Lessons Learned

The development of the Medical Journey Log provided valuable lessons across technical implementation, project planning, documentation, and personal workflow management. Completing this project over two consecutive courses (CST-451 and CST-452) reinforced the importance of iterative development, consistent testing, and maintaining realistic scope while balancing academic, work, and family responsibilities.

7.1 Technical Lessons Learned

Several technical skills were strengthened throughout the project:

- **ASP.NET Razor Pages Structure**
  Building the system using Razor Pages clarified how Page Models manage POST/GET actions, how data flows through handlers, and how validation is enforced through model binding.
- **MySQL Integration**
  Designing and implementing a relational database reinforced concepts of foreign keys, referential integrity, and normalized table structures. Managing schema changes through MySQL Workbench also improved understanding of database lifecycle management.
- **CRUD Patterns and Guard Logic**
  Implementing full create, read, update, and delete workflows improved proficiency in server-side validation, relational constraints, and ensuring safe data handling through guarded deletes.
- **UI Consistency and Razor Syntax**
  Using shared layouts and consistent .cshtml structure emphasized the importance of clear interface design and clean navigation for end-users.
- **Iterative Testing and Stability**
  Conducting regression tests after each major update highlighted the value of catching issues early and preventing regressions in user workflows.

7.2 Project Management Lessons Learned

The project reinforced several broader development practices:

- **Importance of Requirements Traceability**
  Maintaining clear linkage between requirements, design artifacts, code, and test cases helped ensure nothing was overlooked and provided a smooth path to Milestone 6.
- **Breaking Work Into Iterations**
  Completing Iteration 1 (core features), Iteration 2 (CRUD expansion), and Iteration 3 (testing and polish) taught the importance of focusing on one set of goals at a time.

- **Time Management and Planning**
  Balancing full-time work, family responsibilities, and academic deadlines showed how critical it is to plan ahead, limit scope, and avoid trying to do too much in one iteration.
- **Documentation Discipline**
  Producing milestone deliverables emphasized the value of writing clear summaries, diagrams, and evidence that accurately reflect actual implementation.

7.3 Personal and Professional Growth

- **Confidence in Full-Stack Development**
  Building an end-to-end system increased confidence working with databases, server-side logic, and UI layers together.
- **Problem-Solving Under Constraints**
  Deciding not to implement authentication due to scope and personal-use context reinforced the importance of realistic planning and avoiding feature creep.
- **Communication Through Documentation**
  Creating structured milestone reports, test plans, and a poster presentation improved professional communication skills and showcased the ability to present technical work clearly.

7.4 Christian Worldview Reflection

From a Christian worldview perspective, this project reflects themes of stewardship, service, and compassion. The application was inspired by the real needs of a family managing medical challenges, and the development process emphasized:

- Using technical skills to support others
- Exercising diligence and integrity in work
- Creating a tool that promotes care, organization, and clarity during stressful situations

This reinforces the idea that technology can be used not only for efficiency but also as a means of supporting the well-being of families and demonstrating love through thoughtful design.

8. Future Enhancements

Although the current version of the Medical Journey Log meets 100% of the defined requirements and functions reliably for local caregiver use, several enhancements could strengthen the system in future iterations. These improvements would expand usability, increase flexibility, and support more robust long-term medical tracking.

8.1 Expanded Child and Symptom Features

Future versions could include additional fields and customization options such as:

- Profile photos for children
- Additional medical history fields (diagnoses, allergies, baseline notes)
- Custom symptom categories or severity scales
- Ability to attach documents or images to symptom entries

These enhancements would make the system even more useful for caregivers managing complex medical information.

8.2 Reporting and Export Tools

While the current system provides detailed symptom logs, caregivers could benefit from advanced reporting features:

- Printable or downloadable PDF summaries
- Date-range filtering for symptoms
- Charts showing symptom history trends
- Export to CSV for sharing with doctors or therapists

These enhancements would make the system more applicable in professional medical environments.

8.3 Improved UI and Accessibility

The existing UI is functional and clean, but future versions could incorporate:

- Mobile-responsive improvements for use on phones and tablets
- Color-coded symptom indicators
- Enhanced layout with cards, badges, or icons
- Accessibility updates following WCAG guidelines

These improvements would enhance readability and usability for caregivers under stress.

8.4 Integration and Backup Features

To support more advanced usage, future iterations could include:

- Automatic data backup to a secure location
- Optional integration with cloud platforms (Azure, AWS, Supabase, etc.)
- Optional caregiver accounts with passwords and multi-device support
- Version history for symptom entries (audit log)

These features would extend the system beyond the local environment.

8.5 Performance and Scalability Enhancements

While the current dataset is small, performance-focused improvements could be introduced:

- Database indexing for large symptom histories
- Pagination for children or symptom lists
- Caching frequently accessed data
- Optimizing queries for large datasets

These optimizations would support long-term use as symptom logs grow.

8.6 Additional Testing and Quality Assurance

Future testing improvements could include:

- Larger-scale dataset testing to measure performance
- Edge-case tests (extreme date ranges, large text inputs, rapid sequential updates)
- Automated test suites using xUnit or Selenium
- Load testing for database operations

These enhancements would further improve reliability and robustness.

9. Final Project Status

The Medical Journey Log project is fully complete and meets all functional and non-functional requirements defined during the Analysis Phase. Over the course of CST-451 and CST-452, the system progressed through the Planning, Requirements, Design, Implementation, and Testing phases, resulting in a stable, reliable, and fully documented application.

9.1 Completion Summary

- **Requirements Completed:** 100%
- **Core Features Implemented:**
  - Add, view, edit, and delete child profiles
  - Log, edit, and delete symptoms
  - Guarded delete logic to prevent data loss
  - Structured Razor Pages UI
  - Full MySQL relational database design
  - Validation for all forms and inputs
- **Design Artifacts Completed:**
  - Architecture diagrams
  - Database schema
  - Navigation flow
  - Wireframes
- **Testing Artifacts Completed:**
  - Unit tests
  - Integration tests
  - System tests
  - Regression testing (Iteration 3)
  - Complete test plan and test case documentation
- **Documentation Completed:**
  - All Milestones 1–6
  - Technology Showcase poster
  - Final project review (this document)

9.2 System Quality Assessment

The final build demonstrates:

- **Strong stability** across all workflows
- **Consistent UI experience** aligned with wireframes
- **Clean and maintainable code** following Razor Pages best practices
- **Accurate data integrity** enforced through validation and database constraints
- **Reliable CRUD operations** with robust guarded logic

- **No outstanding bugs or defects** after regression testing

The system is fully usable for its intended purpose as a personal medical logging tool for caregivers.

9.3 Project Alignment with Goals

All goals outlined in the Project Proposal and carried through the SDLC were successfully met:

- The application supports organized symptom tracking
- Navigation and workflows are consistent and caregiver-friendly
- The MySQL database structure aligns with the relational model planned in design
- The final product accurately reflects the needs of a family managing complex medical conditions

The completed system fulfills the original vision of creating a tool to support caregivers through better documentation, clarity, and organization.

9.4 Final Statement

The project is complete, stable, and ready for submission. All milestones have been met, and the final implementation fully aligns with the requirements, design artifacts, and testing results established throughout CST-451 and CST-452. The Medical Journey Log stands as a functional, reliable, and well-documented capstone application demonstrating mastery of full-stack development concepts.

Appendix A – Technology Showcase Poster

Medical Journey Log: A Family Care Tracker

Alex Frear - Grand Canyon University - College of Engineering and Technology

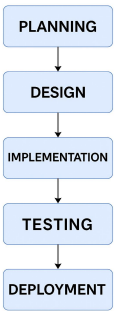
Project Description & Justification

The Medical Journey Log is a web application that helps families track medical notes, symptoms, and milestones for their children. It was designed to simplify communication among caregivers and reduce stress for families managing complex medical conditions. The project was inspired by my own family's experiences with scheduling, treatments, and therapy tracking for my son.

Functional Requirements & Specifications

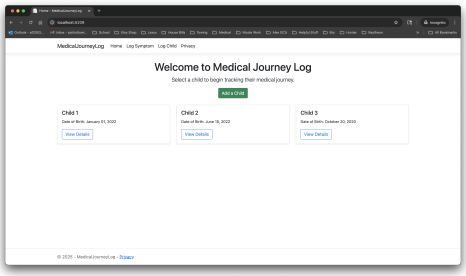
- Caregiver login (private, single-user design)
- Add and manage child profiles
- Log symptoms, medications, appointments, and milestones
- Responsive interface built with ASP.NET Razor Pages and C#
- MySQL backend with normalized relational design
- Accessible design following WCAG guidelines
- Data validation and form feedback

Design Process Overview

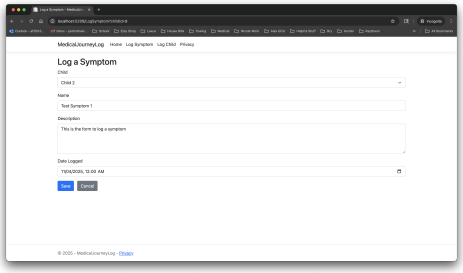


The development of the *Medical Journey Log* followed a structured Software Development Life Cycle (SDLC) to ensure organization, consistency, and quality throughout the project. The process began with **Planning**, where project goals and user needs were clearly defined. During the **Design** phase, system architecture diagrams, database structures, and user interface wireframes were created to guide implementation. The **Implementation** phase focused on coding the ASP.NET Razor Pages and integrating MySQL database functionality. Once the system was operational, **Testing** was conducted to verify that features performed correctly and that the application met all functional requirements. Finally, the **Deployment** phase prepared the application for presentation and demonstration at the Technology Showcase.

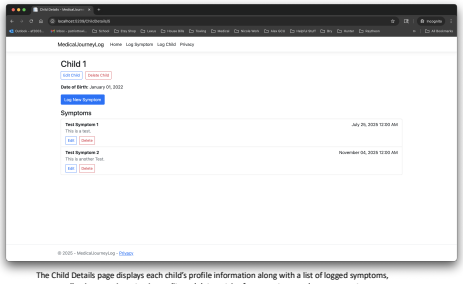
Relevant Data Summaries



The Medical Journey Log home page displays a list of children profiles, allowing caregivers to select or add a child to begin tracking their medical journey.



The Log a Symptom page allows caregivers to record and describe a child's symptoms, providing details such as name, description, and date for accurate medical tracking.



The Child Details page displays each child's profile information along with a list of logged symptoms, allowing caregivers to view, edit, or delete entries for accurate record management.

GCU COLLEGE OF ENGINEERING & TECHNOLOGY

Project developed under CST-452: Senior Project II, Grand Canyon University.